ISO/IEC/IEEE 29148:2018

Software Requirements Specification

For **PaySlip Pro**

Version 0.1

Author: **Farhan Aslam**

Date: **2026-06-01**

Table of Contents

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Employee Salary Slip Automation System, hereinafter referred to as "the System." It is intended to serve as a complete technical reference for developers, system administrators, and stakeholders involved in the deployment and maintenance of the System.

This document conforms to IEEE Std 29148-2018 (Systems and Software Engineering — Life Cycle Processes — Requirements Engineering) and draws on IEEE Std 1016-2009 (Standard for Information Technology — Systems Design — Software Design Descriptions) for architectural documentation.

1.2 Scope

The System is a full-stack web application that automates the monthly payroll slip distribution process within an organisation. It accepts structured payroll data in CSV or Excel format, generates individual PDF salary slips for each employee, validates recipient email addresses at the DNS level, and delivers those slips via Gmail SMTP.

The System is not responsible for:

- Computing raw payroll figures (salary calculations must be finalised in the uploaded file).
- Employee authentication or access control (the application currently assumes single-tenant internal use).
- Archiving historical PDFs beyond Cloudinary storage.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface
CSV	Comma-Separated Values
DNS	Domain Name System
HRA	House Rent Allowance
MX Record	Mail Exchange Record — a DNS record specifying a mail server for a domain
ORM	Object-Relational Mapper
PDF	Portable Document Format
SRS	Software Requirements Specification
SMTP	Simple Mail Transfer Protocol
SSR	Server-Side Rendering

1.4 References

Reference	Source
IEEE Std 29148-2018	IEEE, Systems and Software Engineering — Life Cycle Processes — Requirements Engineering

IEEE Std 1016-2009	IEEE, Standard for IT — Systems Design — Software Design Descriptions
Next.js Documentation	https://nextjs.org/docs
Prisma ORM Documentation	https://www.prisma.io/docs
Nodemailer Documentation	https://nodemailer.com
pdfkit Documentation	https://pdfkit.org

1.5 Overview

Section 2 provides a high-level description of the System and its context. Section 3 describes the system architecture. Sections 4 and 5 define the functional requirements and the complete API contract. Sections 6–10 cover data formats, validation logic, deployment, security, and third-party library inventory.

2. Overall Description

2.1 Product Perspective

The System operates as a standalone, self-contained web application. It interfaces with four external services:

1. **Neon PostgreSQL (via Prisma ORM) —** persistent storage for employee records, salary records, and email dispatch logs.
2. **Cloudinary —** cloud storage for archival copies of uploaded payroll files.
3. **Gmail SMTP (via Nodemailer) —** the email transport layer for delivering salary slips.
4. **DNS Resolver (Node.js built-in) —** used to validate recipient email domains before attempting delivery.

2.2 Product Functions

The System provides the following primary capabilities:

- **Employee Management —** Add, view, and update employee records individually or in bulk via CSV/Excel import.
- **Payroll Upload —** Accept a monthly payroll file, validate all employee references, compute net salary, and persist the records.
- **PDF Generation —** Produce a formatted, A4-sized salary slip PDF per employee.
- **Email Dispatch —** Validate recipient email domain via DNS MX lookup and send the PDF as an attachment.
- **Delivery Logging —** Record the outcome (sent / failed) of every dispatch attempt, including error reasons on failure.
- **Dashboard Analytics —** Display real-time statistics scoped to the current calendar month.

2.3 User Characteristics

The intended users are HR administrators and payroll officers within an organisation. They are assumed to have basic computer literacy — capable of working with spreadsheet software and navigating a web application. No programming knowledge is required.

2.4 Constraints

- The System depends on a valid, active Gmail account with App Passwords enabled.
- Payroll input files must conform to the column formats documented in Section 6.
- The System does not implement user authentication and should be deployed in a private, access-controlled environment.
- PDF rendering uses embedded standard fonts (Helvetica). Custom fonts are not supported in this version.

2.5 Assumptions and Dependencies

- All salary component values in uploaded files are pre-calculated and accurate.

- Employee IDs used in the salary file must already exist in the System before a payroll upload is attempted.
- The Neon PostgreSQL instance uses SSL connections (sslmode=require).
- The deployment environment has outbound network access to Gmail SMTP.

3. System Architecture

3.1 Architectural Overview

The System uses the Next.js App Router architecture, which co-locates server-side API routes with the client-side React interface within a single deployment unit.

```
Browser (React Client)
       |
       |  HTTP / HTTPS
       v
Next.js Application Server
  |- /app                    (React Pages and Layouts)
  |     |- page.js           (Dashboard)
  |     |- employees/        (Employee management UI)
  |     |- upload/           (Payroll upload UI)
  |     `- logs/             (Email log viewer)
  |
  |- /app/api                (Next.js Route Handlers - Server-side only)
  |     |- employees/
  |     |- salary/
  |     |- salary/send/
  |     |- logs/
  |     `- stats/
  |
  `- /app/lib                (Shared Server Utilities)
        |- prisma.js          (Database client singleton)
        |- fileParser.js      (CSV/Excel parsing and validation)
        |- pdfGenerator.js    (PDFKit salary slip builder)
        |- mailer.js          (DNS validation + Nodemailer transport)
        `- cloudinary.js      (File archival upload)
```

External service connections (server-side only):

Service	Protocol	Library
Neon PostgreSQL	TCP / SSL	@prisma/client
Cloudinary	HTTPS REST	cloudinary SDK
Gmail SMTP	SMTP / TLS	nodemailer
DNS Resolver	UDP/TCP	dns/promises (Node.js built-in)

3.2 Component Description

3.2.1 File Parser (app/lib/fileParser.js)

Accepts a raw binary buffer and filename. Detects file type by extension (.csv or .xlsx/.xls) and parses rows using csv-parser or xlsx respectively. Returns a normalised array of row objects. Also exposes validateEmployeeData() and validateSalaryData() which assert required fields are present on each row.

3.2.2 PDF Generator (app/lib/pdfGenerator.js)

Accepts an Employee object and a SalaryRecord object. Uses pdfkit to draw an A4-format salary slip with a dark header band, employee details block, earnings and deductions table, and system-generated footer. Returns a Buffer containing the complete PDF binary.

3.2.3 Mailer (app/lib/mailer.js)

Before sending, performs a DNS MX lookup on the domain extracted from the recipient email address. If the lookup fails or returns no records, an error is thrown immediately, marking the delivery as failed without contacting SMTP. On successful DNS validation, uses a nodemailer transporter configured for Gmail SMTP to send the PDF buffer as an attachment.

3.2.4 Prisma Client (app/lib/prisma.js)

A singleton Prisma client instance initialised with the DATABASE_URL environment variable. Prevents multiple concurrent client instantiations, which is important in Next.js hot-reload environments.

3.3 Data Flow

Payroll Upload and Dispatch flow:

```
User selects file + month/year
         |
         v
POST /api/salary
  |- Parse CSV/Excel buffer
  |- Validate column schema
  |- Verify all Employee IDs exist in DB
  |- Upload raw file to Cloudinary (archival)
  `- Upsert SalaryRecord rows in Neon DB
         |
         v
POST /api/salary/send (triggered by user)
  `- For each SalaryRecord:
        |- Generate PDF (pdfGenerator)
        |- DNS MX lookup on employee email domain
        |     |- FAIL  ->  upsert EmailLog {status: "failed"}
        |     `- PASS  ->  sendMail via Gmail SMTP
        |                     |- FAIL  ->  upsert EmailLog {status: "failed"}
        |                     `- PASS  ->  upsert EmailLog {status: "sent"}
```

3.4 Database Schema

3.4.1 Employee

Column	Type	Constraints
id	String (CUID)	Primary Key
employeeId	String	Unique
name	String	Required
email	String	Required
designation	String	Required
department	String	Optional

dateOfBirth	DateTime	Optional
createdAt	DateTime	Default: now()
updatedAt	DateTime	Auto-updated

3.4.2 SalaryRecord

Unique constraint: (employeeId, month, year) — one record per employee per month.

Column	Type	Constraints
id	String (CUID)	Primary Key
employeeId	String	FK → Employee.employeeId
baseSalary	Float	Required
hra	Float	Required
allowances	Float	Required
deductions	Float	Required
netSalary	Float	Required (computed: base + HRA + allowances − deductions)
month	Int	Required (1–12)
year	Int	Required
createdAt	DateTime	Default: now()

3.4.3 EmailLog

Column	Type	Constraints
id	String (CUID)	Primary Key
salaryRecordId	String	Unique, FK → SalaryRecord.id
employeeId	String	FK → Employee.employeeId
recipientEmail	String	Required
status	String	Default: "pending"; values: sent, failed, pending
errorMessage	String	Optional — populated on failure
sentAt	DateTime	Optional — populated on successful send
createdAt	DateTime	Default: now()

4. Specific Requirements

4.1 Functional Requirements

FR-01: Employee Registration (Single)
The System shall allow a user to register a single employee by providing employeeId, name, email, and designation. The department field is optional. The employeeId must be unique across the system.

FR-02: Employee Registration (Bulk Import)
The System shall accept a CSV or Excel file containing multiple employee records and upsert each record into the database. Existing employees (matched by employeeId) shall be updated rather than duplicated.

FR-03: Payroll File Upload
The System shall accept a CSV or Excel file containing salary components for a specific month and year. The file must reference employee IDs that are already registered in the System. Files referencing unknown employee IDs shall be rejected with an informative error message listing the unknown IDs.

FR-04: Salary Computation
The System shall compute net salary as: netSalary = baseSalary + hra + allowances − deductions

FR-05: PDF Generation
For each salary record, the System shall generate a formatted A4 PDF salary slip containing the employee's details and a breakdown of all salary components.

FR-06: Email Domain Validation
Before attempting SMTP delivery, the System shall perform a DNS MX record lookup on the domain portion of the recipient's email address. If the lookup fails or returns no records, the delivery attempt shall be abandoned and the email log shall be marked as failed with an appropriate error message.

FR-07: Email Dispatch
The System shall send the generated PDF salary slip as an email attachment to the employee's registered email address via Gmail SMTP.

FR-08: Delivery Logging
The System shall create or update an EmailLog record for every dispatch attempt, recording the final status (sent or failed), the timestamp of a successful send, or the error message on failure.

FR-09: Dashboard Statistics
The System shall display the following metrics on the dashboard, all scoped to the current calendar month and year: (1) Total registered employees, (2) Number of salary slips generated

this month, (3) Number of emails successfully sent this month, (4) Number of emails that failed this month.

FR-10: Email Log Viewer
The System shall provide a filterable table of all email dispatch records, displaying employee name, pay period, recipient address, status, and dispatch timestamp.

4.2 External Interface Requirements

4.2.1 User Interface
The System shall provide a responsive dark-themed web interface accessible at http://localhost:3000 in development and via a Vercel-hosted URL in production. Navigation is provided via a persistent left sidebar.

4.2.2 Hardware Interfaces
No direct hardware interfaces. The System communicates with all external services over standard TCP/IP.

4.2.3 Software Interfaces
- **Neon PostgreSQL:** Connection via DATABASE_URL (PostgreSQL connection string with SSL).
- **Gmail SMTP:** Connection using credentials in GMAIL_USER and GMAIL_APP_PASSWORD.
- **Cloudinary REST API:** Authenticated using CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, and CLOUDINARY_API_SECRET.

4.3 Non-Functional Requirements
NFR-01: Performance
Each salary slip dispatch batch (up to 100 employees) shall complete within 120 seconds under normal network conditions.

NFR-02: Reliability
A failure to send an individual employee's email shall not interrupt the dispatch process for the remaining employees in the batch.

NFR-03: Data Integrity
All salary record writes shall use upsert operations to prevent duplicate records for the same employee and pay period.

NFR-04: Maintainability
All API route handlers shall log errors to the server console with the originating endpoint clearly identified.

5. API Specification

All API routes are located under /api/. All responses follow the JSON structure shown below:

```
{
  "success": true | false,
  "message": "Human-readable result description",
  "data": { ... }
}
```

5.1 Employee Endpoints

GET /api/employees

Returns all registered employees ordered by creation date (newest first). Each record includes a count of associated salary records.

POST /api/employees — Single Employee (JSON body)

Request body:

```
{
  "employeeId": "EMP001",
  "name": "Jane Doe",
  "email": "jane.doe@company.com",
  "designation": "Software Engineer",
  "department": "Engineering"
}
```

Field	Required	Type
employeeId	Yes	String
name	Yes	String
email	Yes	String
designation	Yes	String
department	No	String

- HTTP 201: Employee created.
- HTTP 400: Missing required fields.
- HTTP 409: Employee ID already exists.

POST /api/employees — Bulk Import (multipart/form-data)

Form field: file — CSV or Excel file (see Section 6.1).

- HTTP 200: All employees imported.
- HTTP 400: File missing or validation errors.

5.2 Salary Endpoints

GET /api/salary

Optional query parameters: month, year, employeeId.

Response data: Array of SalaryRecord objects including linked employee and email log status.

POST /api/salary — Upload Payroll File (multipart/form-data)
Form fields: file (CSV/Excel), month (Integer 1–12), year (Integer).

- HTTP 200: Records upserted successfully.
- HTTP 400: Missing file, invalid month/year, validation errors, or unknown employee IDs.

POST /api/salary/send — Trigger Email Dispatch
Request body:

```
{
  "month": 5,
  "year": 2026
}
```

- HTTP 200: Batch completed. Response includes sent count, failed count, and errors array.
- HTTP 400: Missing month or year.
- HTTP 404: No salary records found for the given period.

5.3 Email Log Endpoints

GET /api/logs
Optional query parameters: status (sent | failed | pending), month, year.

Response data: Array of EmailLog objects including employee name and salary period.

5.4 Dashboard Statistics Endpoint

GET /api/stats
Returns all metrics scoped to the current calendar month and year (determined server-side at request time).

Response data:

```
{
  "totalEmployees": 42,
  "totalSlipsThisMonth": 42,
  "emailsSent": 40,
  "emailsFailed": 2,
  "emailsPending": 0,
  "recentLogs": [ ... ]
}
```

recentLogs contains the 5 most recent EmailLog records ordered by creation date descending.

6. Data Requirements

6.1 CSV Input Format: Employees

Used with POST /api/employees (file upload).

Column Header	Required	Description
employee_id	Yes	Unique employee identifier (e.g., EMP001)
name	Yes	Full name of the employee
email	Yes	Email address for salary slip delivery
designation	Yes	Job title
department	No	Department name

Example:

```
employee_id,name,email,designation,department
EMP001,Alice Smith,alice@company.com,Software Engineer,Engineering
EMP002,Bob Jones,bob@company.com,HR Manager,Human Resources
```

6.2 CSV Input Format: Salary Records

Used with POST /api/salary (file upload).

Column Header	Required	Description
employee_id	Yes	Must match a registered employee ID
base_salary	Yes	Base monthly salary (numeric)
hra	Yes	House Rent Allowance (numeric)
allowances	Yes	Other allowances (numeric)
deductions	Yes	Total deductions (numeric)

Net salary is computed by the system: netSalary = base_salary + hra + allowances − deductions

Example:

```
employee_id,base_salary,hra,allowances,deductions
EMP001,80000,20000,10000,5000
EMP002,95000,25000,12000,6000
```

7. Error Handling and Validation

Rule	Behaviour on Failure
Missing required CSV columns	Request rejected; error list returned
employee_id in salary file not found in DB	Request rejected; missing IDs listed
month outside range 1–12	Request rejected with HTTP 400
Email has no @ symbol	Email log marked failed; batch continues
Email domain has no MX records in DNS	Email log marked failed; batch continues
Gmail SMTP rejects delivery	Email log marked failed; error message stored
Cloudinary upload fails	Warning logged; operation continues (non-blocking)
Duplicate (employeeId, month, year) salary record	Existing record updated (upsert)
Duplicate employeeId on employee creation (JSON)	HTTP 409 returned

On any failure, all API endpoints return:

```
{
  "success": false,
  "message": "A description of what went wrong",
  "error": "The raw error message"
}
```

8. Deployment and Configuration

8.1 Environment Variables

All secrets and configuration values are provided through environment variables. No values are hardcoded in the source. A template file (.env.example) is included in the repository with placeholder values.

Variable	Description
DATABASE_URL	Full PostgreSQL connection string for Neon, including ?sslmode=require
CLOUDINARY_CLOUD_NAME	Cloudinary account cloud name
CLOUDINARY_API_KEY	Cloudinary API key
CLOUDINARY_API_SECRET	Cloudinary API secret
GMAIL_USER	Gmail address used as the SMTP sender
GMAIL_APP_PASSWORD	16-character Gmail App Password (not the account password)

8.2 Local Setup Procedure

Prerequisites: Node.js v18 or later, npm.

5. **Install dependencies:** npm install
6. **Configure environment:** Copy .env.example to .env and fill in all values.
7. **Push database schema:** npx prisma db push
8. **Start development server:** npm run dev (available at http://localhost:3000)

8.3 Vercel Deployment

9. Push the repository to GitHub.
10. Log in to vercel.com and click Add New Project.
11. Import the GitHub repository.
12. Under Environment Variables, add all six variables from Section 8.1.
13. Click Deploy.

The package.json postinstall script (prisma generate) will run automatically during the Vercel build, ensuring the Prisma client is generated for the deployment environment. The default Vercel build command (npm run build) is sufficient — no additional configuration is required.

9. Security Considerations

Area	Consideration
Secrets Management	All credentials are stored as environment variables. The .env file is listed in .gitignore and must never be committed to version control.
Email Credentials	A Gmail App Password is used rather than the primary account password. This limits the scope of credential exposure if compromised.
DNS Validation	Email domain validation via MX lookup reduces the risk of sending to invalid addresses, lowering the likelihood of the sender domain being flagged as spam.
SQL Injection	All database queries are constructed via the Prisma ORM using parameterised operations. Raw SQL is not used anywhere in the codebase.
Access Control	The application does not currently implement user authentication. It is strongly recommended to restrict access at the infrastructure level before deploying to production.
File Uploads	Uploaded files are only parsed in-memory and then forwarded to Cloudinary. No uploaded files are written to the local filesystem.

10. Appendix

10.1 Third-Party Libraries

Library	Version	Purpose
next	16.2.6	Full-stack React framework (App Router)
react	19.2.4	UI component library
@prisma/client	^7.8.0	Type-safe database client (ORM)
prisma	^7.8.0	Database schema management and migration tooling
@prisma/adapter-pg	^7.8.0	PostgreSQL adapter for Prisma
pg	^8.21.0	PostgreSQL driver for Node.js
pdfkit	^0.18.0	PDF document generation
nodemailer	^8.0.10	Email transport via SMTP
cloudinary	^2.10.0	Cloud media storage and delivery
csv-parser	^3.2.1	Streaming CSV file parser
xlsx	^0.18.5	Excel file parser (.xlsx, .xls)
streamifier	^0.1.1	Converts Buffer to readable stream (Cloudinary upload)
dotenv	^17.4.2	Loads .env variables into process.env